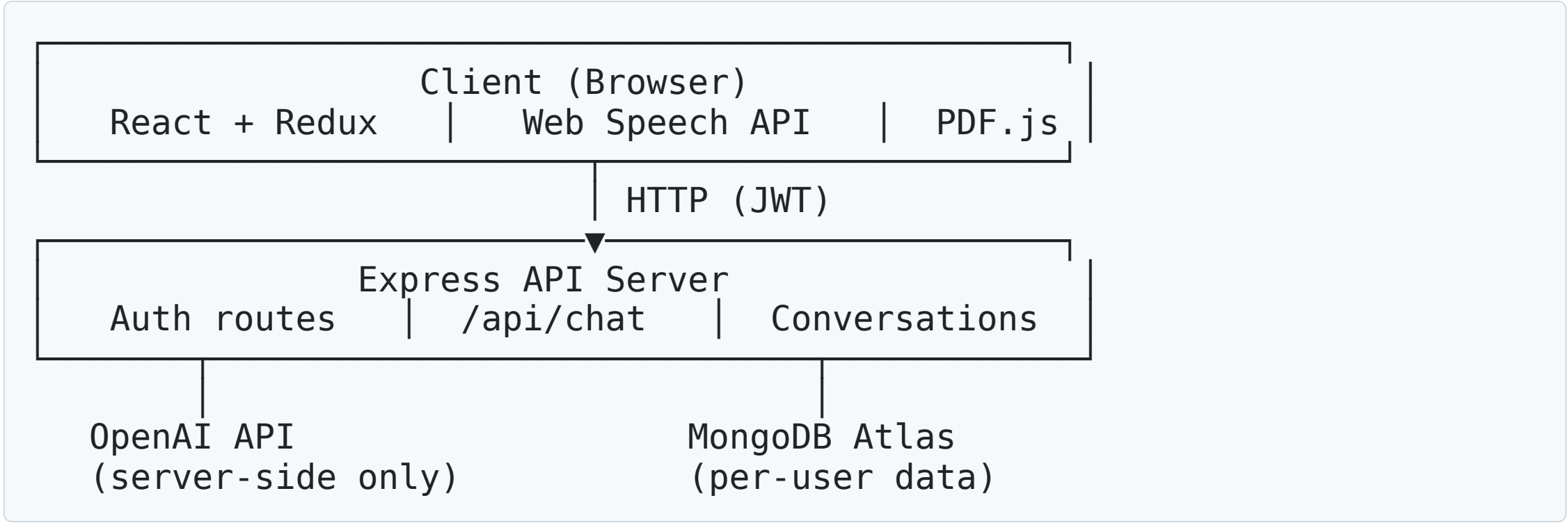
ChatGPT Clone — Architecture & Design

Technical Design Document

System Overview



The client never communicates with OpenAI directly.

All AI requests are proxied through the Express server.

Frontend Architecture

React 18 — component-based UI

Redux Toolkit — global state management

Vite 4 — dev server and build tool

```
App.jsx
 ├── LoginPage.jsx          (auth gate)
 └── Dashboard/
      ├── Sidebar/
      │    ├── NewChatButton
      │    ├── ListItem (per conversation)
      │    └── DeleteConversationsButton
      └── Chat/
           ├── ChatSettings   (model + persona)
           ├── Messages       (message list)
           ├── Message        (single bubble + TTS)
           └── NewMessageInput (text + file + mic)
```

Redux State Shape

```
store
├── auth
│   ├── token        (JWT string | null)
│   └── user         ({ id, email } | null)
└── dashboard
    ├── conversations []
    │   └── { id, messages[], model, persona }
    ├── selectedConversationId
    ├── loading
    └── error
```

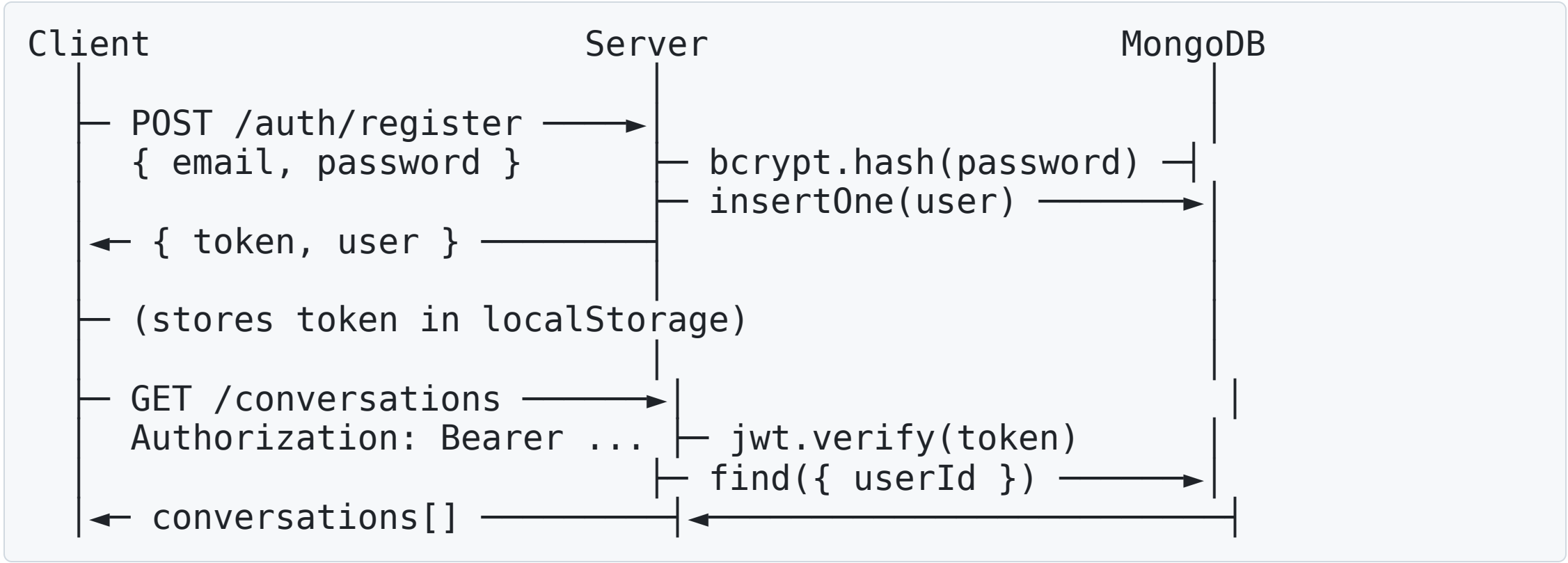
Listener middleware auto-saves any conversation change to the cloud (new message, model change, persona change, new conversation).

Backend Architecture

Express 5 server (`server.js`) — single file, 6 routes

Route	Auth	Purpose
GET /api/health	None	Startup readiness check
POST /api/auth/register	None	Create account (bcrypt hash)
POST /api/auth/login	None	Verify password, issue JWT
POST /api/chat	JWT	Proxy request to OpenAI
GET /api/conversations	JWT	Load user's conversations
POST /api/conversations/:id	JWT	Upsert one conversation
DELETE /api/conversations	JWT	Delete all for this user

Authentication Flow



Token expiry: **7 days**. Passwords never stored in plaintext.

AI Request Flow

```
User types message and clicks Send
          |
          ▼
NewMessageInput.jsx
  dispatches sendConversationMessage (Redux thunk)
          |
          ▼
openaiService.js
  POST /api/chat  { messages[], model }
  Authorization: Bearer <JWT>
          |
          ▼
server.js  /api/chat
  jwt.verify(token)  →  authorized
  openai.createChatCompletion({ model, messages })
          |
          ▼
OpenAI API  (key never leaves server)
          |
          ▼
server.js returns { content: "..." }
          |
          ▼
Redux store  →  addMessage(aiMessage)
  listener middleware  →  apiSaveConversation()
```

File Upload Pipeline

```
User selects PDF or TXT file
          |
          ▼
fileExtractor.js
  TXT  →  FileReader.readAsText()
  PDF  →  pdfjs-dist  →  extract text from each page
  Truncate at 50,000 characters
          |
          ▼
NewMessageInput stores { name, text } in local state
          |
User clicks Send
          |
          ▼
Message content = "[File: name.pdf]\n{text}\n\n---\n\n{userQuestion}"
Stored with: { fileName, userText }  (for display)
Full content sent to OpenAI
```

File content stays client side — never uploaded to our server directly

Voice I/O Architecture

Speech-to-Text (input)

```
User clicks mic button
  → SpeechRecognition.start()
  → onresult fires as words are spoken
  → interim results update input field live
  → onend fires on silence → listening stops
```

Text-to-Speech (output)

```
User clicks speaker button on AI message
  → SpeechSynthesisUtterance(content)
  → window.speechSynthesis.speak(utterance)
  → onend callback resets button state
  → User can click again to cancel via speechSynthesis.cancel()
```

Zero external packages — uses built-in browser APIs only.

Data Model

User document (MongoDB)

```
{
  "_id": "uuid",
  "email": "user@example.com",
  "passwordHash": "$2b$10$...",
  "createdAt": "2026-01-14T..."
}
```

Conversation document (MongoDB)

```
{
  "_id": "uuid",
  "userId": "uuid",
  "model": "gpt-4o-mini",
  "persona": "You are a helpful assistant.",
  "messages": [
    { "id": "uuid", "content": "Hello", "aiMessage": false,
```

Security Design

Threat	Mitigation
API key exposure	Key is server-only env var; no VITE_ prefix
Unauthorized API access	All AI/data routes require valid JWT
Password storage	bcryptjs with salt rounds = 10
Cross-user data access	All DB queries filter by userId from JWT
Secret leakage via git	.env, .env.local, *.pem all in .gitignore
Large payload abuse	express.json limit = 4 MB

Technology Decisions

Decision	Choice	Reason
State management	Redux Toolkit	Predictable, DevTools support
Database	MongoDB Atlas	Flexible schema, free tier, managed
Auth	JWT (stateless)	No session storage needed server-side
Build tool	Vite 4	Fast HMR, ESM-native
PDF parsing	pdfjs-dist	Browser-native, no server upload
Voice	Web Speech API	Zero dependencies, built into Chrome/Edge

Testing Strategy

Type	Location	Coverage
Unit	`src/__tests__/unit/`	Redux reducers, speech detection, auth state
Integration	`src/__tests__/integration/`	Combined store, multi-action flows
Regression	`src/__tests__/regression/`	Message/conversation structural invariants
Acceptance	`src/__tests__/acceptance/`	LoginPage render, form interactions

Run all tests: `npm test`

44 tests — 100% pass rate